Lab 3:

- Image gallery:
- Prompt: Tôi là sinh viên đang học môn lập trình ứng dụng di động bằng Android Studio. Hiện tại tôi đang làm bài homework trong Lab 3: “Image Gallery with Object Detection”.
-
- Project của tôi là project Android hiện có, không tạo project mới. Tôi muốn bạn hỗ trợ implement bài này trực tiếp trong project hiện tại.
-
- Yêu cầu chính:
- - Code bằng Java.
- - Dùng XML layout truyền thống.
- - Không dùng Jetpack Compose.
- - Không chuyển toàn bộ project sang Kotlin.
- - Nếu project có sẵn Kotlin thì vẫn giữ nguyên, nhưng phần bài này ưu tiên viết Java.
- - Code đơn giản, rõ ràng, dễ hiểu, phù hợp trình độ sinh viên đang học Android.
- - App cần build và chạy được.
-
- Mục tiêu của app:
- Tạo một app Image Gallery lấy ảnh từ Pixabay API, hiển thị ảnh dạng grid, sau đó dùng ML Kit để nhận diện nội dung/object trong ảnh.
-
- Các chức năng cần có:
-
- 1. Search ảnh từ Pixabay API
- - Người dùng nhập từ khóa vào ô search.
- - Nếu người dùng không nhập gì thì mặc định search “nature”.
- - App gọi Pixabay API để lấy danh sách ảnh.
-
- 2. Hiển thị ảnh
- - Dùng RecyclerView dạng Grid, 2 cột.
- - Load ảnh bằng Glide.
- - Mỗi item hiển thị ảnh, tags, tên user và label AI.
-
- 3. Phân trang đơn giản
- - page bắt đầu từ 1.
- - per_page = 20.
- - Khi scroll gần cuối danh sách thì tự load trang tiếp theo.
- - Tránh gọi load nhiều lần cùng lúc.
-
- 4. Object detection / image labeling
- - Dùng Google ML Kit Image Labeling để nhận diện nội dung ảnh.
- - Sau khi ảnh được lấy từ Pixabay, app xử lý ảnh và hiển thị label bên dưới ảnh.
- - Ban đầu label hiển thị “Processing...”.

- - Khi xử lý xong thì hiển thị tối đa 3 label.
- - Nếu lỗi hoặc không nhận diện được thì hiển thị “Unknown”.
-
- 5. AI Cache
- - Nếu một ảnh đã được xử lý AI rồi thì không xử lý lại.
- - Cache kết quả bằng image ID hoặc image URL.
- - Chỉ cần dùng HashMap đơn giản, không cần Room database.
-
- Thông tin Pixabay API:
- Base URL:
- https://pixabay.com/
-
- Endpoint:
- GET /api/
-
- Query parameters:
- - key: API key Pixabay
- - q: từ khóa search
- - image_type: photo
- - page: số trang
- - per_page: số ảnh mỗi trang
-
- Ví dụ:
- https://pixabay.com/api/?key=YOUR_API_KEY&q=flower&image_type=photo&page=1&per_page=20
-
- Tạo file Constants.java:
- public class Constants {
- public static final String PIXABAY_BASE_URL = "https://pixabay.com/";
- public static final String PIXABAY_API_KEY = "YOUR_PIXABAY_API_KEY_HERE";
- public static final int PAGE_SIZE = 20;
- }
-
- Thêm comment rõ rằng tôi cần thay YOUR_PIXABAY_API_KEY_HERE bằng API key thật lấy từ Pixabay.
-
- Dependencies cần dùng:
- - Retrofit
- - Gson Converter
- - RecyclerView
- - Glide
- - Material nếu cần
- - ML Kit Image Labeling
-

- Nếu project dùng build.gradle Groovy thì thêm dạng:
- implementation 'com.squareup.retrofit2:retrofit:2.9.0'
- implementation 'com.squareup.retrofit2:converter-gson:2.9.0'
- implementation 'androidx.recyclerview:recyclerview:1.3.2'
- implementation 'com.github.bumptech.glide:glide:4.16.0'
- annotationProcessor 'com.github.bumptech.glide:compiler:4.16.0'
- implementation 'com.google.android.material:material:1.11.0'
- implementation 'com.google.mlkit:image-labeling:17.0.8'
-
- Nếu project dùng build.gradle.kts thì thêm dạng Kotlin DSL tương ứng.
- Nếu dependency nào đã có rồi thì không thêm trùng.
-
- AndroidManifest:
- Thêm quyền Internet nếu chưa có:
- <uses-permission android:name="android.permission.INTERNET" />
-
- Cấu trúc file gợi ý:
- - data/model/PixabayResponse.java
- - data/model/ImageItem.java
- - data/model/ProcessedImage.java
- - data/remote/PixabayApiService.java
- - data/remote/RetrofitClient.java
- - data/repository/ImageRepository.java
- - ml/ImageProcessor.java
- - ml/AiCache.java
- - ui/MainActivity.java
- - ui/ImageAdapter.java
- - util/Constants.java
-
- Lưu ý:
- Hãy dùng đúng package hiện tại của project. Nếu package hiện tại khác với ví dụ thì sửa package cho đúng.
-
- Data model cần có:
-
- PixabayResponse.java:
- - int total
- - int totalHits
- - List<ImageItem> hits
-
- ImageItem.java:
- Các field cần lấy từ Pixabay:
- - int id
- - String pageURL
- - String type

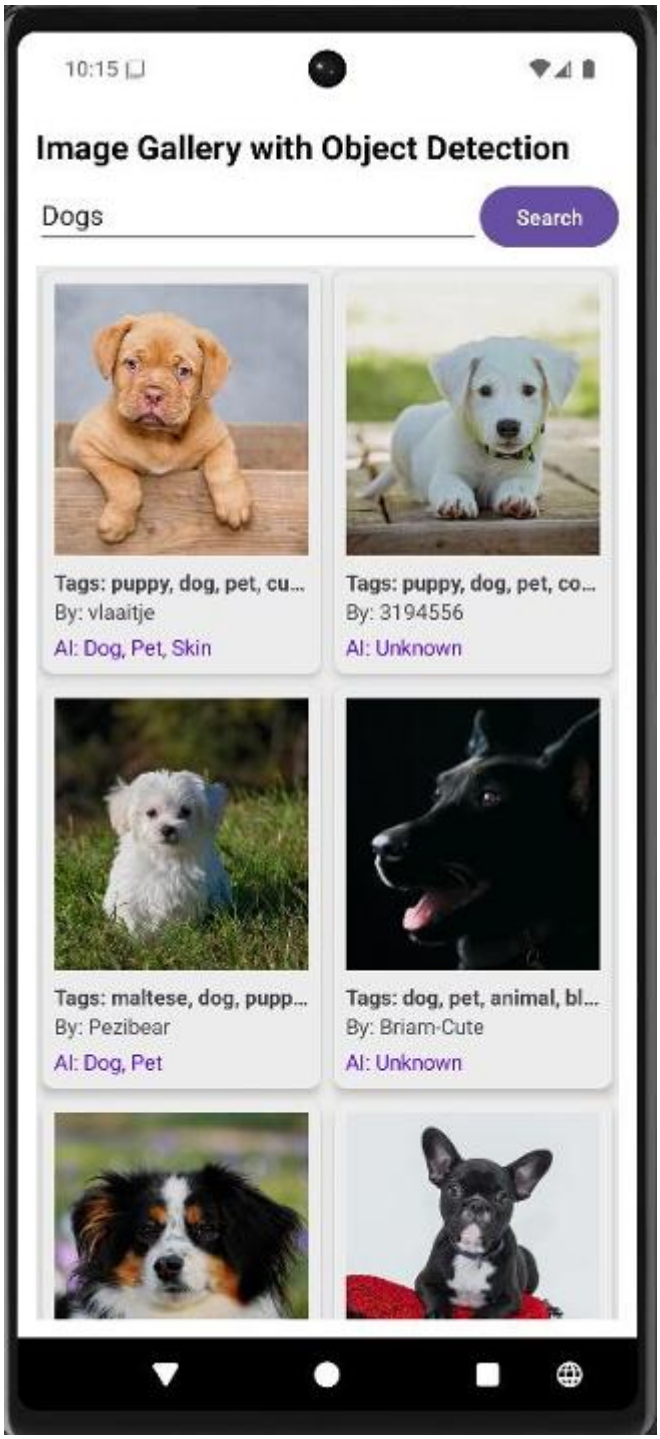
- - String tags
- - String previewURL
- - String webformatURL
- - String largeImageURL
- - int imageWidth
- - int imageHeight
- - int likes
- - int views
- - int downloads
- - String user
-
- ProcessedImage.java:
- - ImageItem imageItem
- - List<String> labels
- - boolean processed
-
- Retrofit API:
- Tạo PixabayApiService.java:
-
- public interface PixabayApiService {
- @GET("api/")
- Call<PixabayResponse> searchImages(
- @Query("key") String apiKey,
- @Query("q") String query,
- @Query("image_type") String imageType,
- @Query("page") int page,
- @Query("per_page") int perPage
-);
- }
-
- Tạo RetrofitClient.java:
- - Dùng Constants.PIXABAY_BASE_URL.
- - Dùng GsonConverterFactory.
- - Tạo Retrofit singleton.
-
- Repository:
- Tạo ImageRepository.java để gọi API.
-
- Method cần có:
- public void searchImages(
- String query,
- int page,
- int perPage,
- ImageCallback callback
-)

-
- Interface:
- public interface ImageCallback {
- void onSuccess(List<ImageItem> images, int totalHits);
- void onError(String errorMessage);
- }
-
- AI Cache:
- Tạo AiCache.java dùng HashMap đơn giản:
-
- public class AiCache {
- private final Map<String, List<String>> cache = new HashMap<>();
-
- public boolean contains(String key) {
- return cache.containsKey(key);
- }
-
- public List<String> get(String key) {
- return cache.get(key);
- }
-
- public void put(String key, List<String> labels) {
- cache.put(key, labels);
- }
-
- public void clear() {
- cache.clear();
- }
- }
-
- ImageProcessor:
- Tạo ImageProcessor.java.
- Dùng Glide tải ảnh thành Bitmap, sau đó dùng ML Kit Image Labeling để xử lý.
-
- Logic:
- - Lấy URL từ imageItem.getWebformatURL().
- - Nếu URL null thì trả về “Unknown”.
- - Dùng Glide tải Bitmap trong background thread.
- - Tạo InputImage từ Bitmap.
- - Dùng ImageLabeling.getClient(ImageLabelerOptions.DEFAULT_OPTIONS).
- - Lấy tối đa 3 label.
- - Ưu tiên label có confidence >= 0.5.
- - Nếu không có label thì trả về “Unknown”.
- - Không để app crash nếu ảnh lỗi hoặc ML Kit lỗi.
-

- MainActivity:
- Tạo hoặc sửa MainActivity.java.
-
- Các view cần có:
- - EditText edtSearch
- - Button btnSearch
- - RecyclerView recyclerViewImages
- - ProgressBar progressBar
- - TextView tvError
- - TextView tvEmpty
-
- Các biến chính:
- - ImageRepository repository
- - ImageProcessor imageProcessor
- - AiCache aiCache
- - ImageAdapter adapter
- - List<ProcessedImage> allImages = new ArrayList<>()
- - int currentPage = 1
- - boolean isLoading = false
- - boolean isLastPage = false
- - String currentQuery = "nature"
- - int totalHits = 0
-
- Behavior:
- - Khi mở app thì tự search “nature”.
- - Khi bấm Search:
- + lấy keyword từ EditText
- + nếu rỗng thì dùng “nature”
- + reset danh sách
- + currentPage = 1
- + isLastPage = false
- + load lại ảnh
- - Khi load ảnh thành công:
- + thêm ảnh vào danh sách
- + label ban đầu là “Processing...”
- + gọi processImageWithCache()
- + update RecyclerView
- - Khi scroll gần cuối thì load page tiếp theo.
- - Nếu API lỗi thì hiện tvError.
- - Nếu không có ảnh thì hiện tvEmpty.
-
- Smart search:
- Implement đơn giản bằng method normalizeQuery(String query):
- - trim
- - lowercase

- - nếu rỗng thì trả “nature”
- - nếu có nhiều khoảng trắng thì xử lý gọn lại
- - không cần làm AI phức tạp
-
- Adapter:
- Tạo ImageAdapter.java extends RecyclerView.Adapter.
-
- Mỗi item hiển thị:
- - ImageView imgPhoto
- - TextView tvTags
- - TextView tvUser
- - TextView tvLabels
-
- Trong onBindViewHolder:
- - Load ảnh bằng Glide.
- - Hiển thị:
- + Tags: ...
- + By: ...
- + AI: label1, label2, label3
- - Nếu đang xử lý thì hiện “AI: Processing...”.
-
- Layout XML:
- Tạo hoặc sửa activity_main.xml.
-
- Root LinearLayout vertical, padding 12dp.
- Gồm:
- - TextView title: “Image Gallery with Object Detection”
- - LinearLayout horizontal chứa EditText và Button Search
- - ProgressBar
- - TextView tvError
- - TextView tvEmpty
- - RecyclerView chiếm phần còn lại màn hình
-
- Tạo item_image.xml:
- - Root có thể dùng LinearLayout hoặc CardView.
- - ImageView cao khoảng 180dp, scaleType centerCrop.
- - TextView tags
- - TextView user
- - TextView AI labels
-
- Placeholder:
- Nếu chưa có thì tạo res/drawable/image_placeholder.xml màu xám nhạt để dùng cho Glide placeholder/error.
-
- Xử lý lỗi:

- App không được crash khi:
- - API key sai
- - Không có Internet
- - Pixabay trả danh sách rỗng
- - Image URL null
- - Glide load ảnh lỗi
- - ML Kit xử lý lỗi
- - Người dùng search keyword rỗng
- - Scroll gọi load nhiều lần
-
- Sau khi làm xong, hãy báo lại:
- 1. Danh sách file đã tạo hoặc sửa.
- 2. Dependency đã thêm.
- 3. Vị trí cần nhập API key: Constants.PIXABAY_API_KEY.
- 4. Cách chạy app:
- - Lấy API key từ Pixabay.
- - Thay YOUR_PIXABAY_API_KEY_HERE bằng API key thật.
- - Sync Gradle.
- - Run app.
- - Nhập keyword để search ảnh.
- 5. Ghi rõ nếu phần object detection đang dùng ML Kit Image Labeling:
- “This implementation uses ML Kit Image Labeling to identify objects/categories in images.”



*Sentiment app:

- Prompt: Mình là sinh viên đang học lập trình ứng dụng di động bằng Android Studio. Hiện tại mình đang làm một bài Sentiment App đơn giản bằng Java và XML trong project Android có sẵn. Mình muốn bạn hỗ trợ mình implement trực tiếp vào project hiện tại, không tạo project mới và cũng không chuyển project sang Kotlin hay Jetpack Compose.

Trong bài này mình đã có sẵn file model tên sentiment_model.tflite. Model này được tạo từ TensorFlow/Keras, train trên IMDB dataset rồi convert sang TensorFlow Lite. Lưu ý là model

của mình không phải dạng TensorFlow Lite Task TextClassifier, nên không dùng TextClassifier hay NLClassifier. Model này nhận input trực tiếp là String tensor và output ra một giá trị float sigmoid, nên hãy dùng TensorFlow Lite Interpreter để chạy model.

File model cần được đặt ở đường dẫn:

app/src/main/assets/sentiment_model.tflite

Nếu trong project chưa có thư mục assets thì hãy tạo thư mục:

app/src/main/assets

Model hoạt động như sau: input là String tensor có shape [1], output là float sigmoid có shape [1][1]. Nếu output gần 1.0 thì hiểu là Positive, gần 0.0 thì hiểu là Negative, còn ở khoảng giữa thì hiểu là Neutral. Trên giao diện không cần hiển thị score, chỉ cần hiển thị nhãn kết quả. Quy tắc phân loại là: nếu score >= 0.65 thì Positive, nếu score <= 0.35 thì Negative, còn lại là Neutral.

Project này chỉ cần làm app sentiment đơn giản, nên không dùng Retrofit, NewsAPI, Paging hay bất kỳ phần gọi API nào. Mình muốn code Java rõ ràng, dễ hiểu, phù hợp với bài lab sinh viên và quan trọng là app phải build, chạy được.

Trong file app/build.gradle, nếu chưa có thì hãy thêm dependency TensorFlow Lite:

```
implementation 'org.tensorflow:tensorflow-lite:2.14.0'
implementation 'org.tensorflow:tensorflow-lite-select-tf-ops:2.14.0'
```

Không thêm dependency:

```
implementation 'org.tensorflow:tensorflow-lite-task-text:0.4.4'
```

vì model này không chạy theo TextClassifier hoặc NLClassifier.

Bạn hãy tạo hoặc sửa các file chính gồm MainActivity.java, SentimentAnalyzer.java, SentimentResult.java và activity_main.xml. Khi tạo file Java, nhớ dùng đúng package hiện tại của project, không tự đặt package khác làm lỗi project.

Đầu tiên, tạo class SentimentResult.java để lưu kết quả phân tích. Class này chỉ cần có label và errorMessage. Label chỉ nên là một trong các giá trị Positive, Neutral, Negative hoặc Unknown. Có thể viết class như sau:

```
public class SentimentResult {
    private String label;
    private String errorMessage;

    public SentimentResult(String label, String errorMessage) {
        this.label = label;
        this.errorMessage = errorMessage;
    }

    public String getLabel() {
        return label;
    }

    public String getErrorMessage() {
        return errorMessage;
    }
}
```

Tiếp theo, tạo class SentimentAnalyzer.java. Class này sẽ chịu trách nhiệm load file sentiment_model.tflite từ assets và chạy inference bằng org.tensorflow.lite.Interpreter. Trong class này không dùng TextClassifier, không dùng NLClassifier. Class cần có method:

```
public SentimentResult analyze(String text)
```

Trong method analyze, nếu text bị null hoặc người dùng chưa nhập gì thì trả về Unknown kèm message “Please enter text first.”. Nếu model chưa load được thì trả về Unknown kèm message “Model file not found or cannot be loaded.”. Nếu chạy model thành công thì lấy output[0][0] làm score, sau đó phân loại theo rule score >= 0.65 là Positive, score <= 0.35 là Negative, còn lại là Neutral. Nếu quá trình inference bị lỗi thì trả về Unknown kèm message “Cannot analyze this text.”.

Có thể dùng đoạn code mẫu sau trong SentimentAnalyzer.java:

```
private Interpreter interpreter;

private MappedByteBuffer loadModelFile(Context context) throws IOException {
    AssetFileDescriptor fileDescriptor = context.getAssets().openFd("sentiment_model.tflite");
    FileInputStream inputStream = new FileInputStream(fileDescriptor.getFileDescriptor());
    FileChannel fileChannel = inputStream.getChannel();
    long startOffset = fileDescriptor.getStartOffset();
    long declaredLength = fileDescriptor.getDeclaredLength();
    return fileChannel.map(FileChannel.MapMode.READ_ONLY, startOffset, declaredLength);
}
```

Constructor:

```
public SentimentAnalyzer(Context context) {
    try {
        Interpreter.Options options = new Interpreter.Options();
        interpreter = new Interpreter(loadModelFile(context), options);
    } catch (Exception e) {
        interpreter = null;
        Log.e("SentimentAnalyzer", "Cannot load model", e);
    }
}
```

Method analyze:

```
public SentimentResult analyze(String text) {
    if (text == null || text.trim().isEmpty()) {
        return new SentimentResult("Unknown", "Please enter text first.");
    }

    if (interpreter == null) {
        return new SentimentResult("Unknown", "Model file not found or cannot be loaded.");
    }

    try {
        String[] input = new String[]{ text };
        float[][] output = new float[1][1];

        interpreter.run(input, output);

        float score = output[0][0];
        String label;

        if (score >= 0.65f) {
            label = "Positive";
        } else if (score <= 0.35f) {
            label = "Negative";
        } else {
            label = "Neutral";
        }

        return new SentimentResult(label, null);
    } catch (Exception e) {
```

```
        Log.e("SentimentAnalyzer", "Analyze error", e);
        return new SentimentResult("Unknown", "Cannot analyze this text.");
    }
}
```

Nếu lúc chạy interpreter.run(input, output) bị lỗi do input hoặc output shape không khớp với model thật, hãy kiểm tra lại signature của model rồi chỉnh lại cho đúng. Tuy nhiên vẫn giữ nguyên ý tưởng chính là model nhận input dạng String tensor và trả về output dạng float sigmoid.

Sau đó tạo hoặc sửa MainActivity.java. Giao diện chỉ cần đơn giản, gồm một EditText để nhập câu tiếng Anh, một Button để bấm phân tích, một TextView để hiển thị kết quả và một TextView để hiển thị lỗi nếu có. Các view có id lần lượt là edtInput, btnAnalyze, tvResult và tvError.

Khi người dùng bấm nút Analyze, app lấy text từ edtInput rồi gọi analyzer.analyze(text). Kết quả hiển thị theo dạng:

```
tvResult.setText("Sentiment: " + result.getLabel());
```

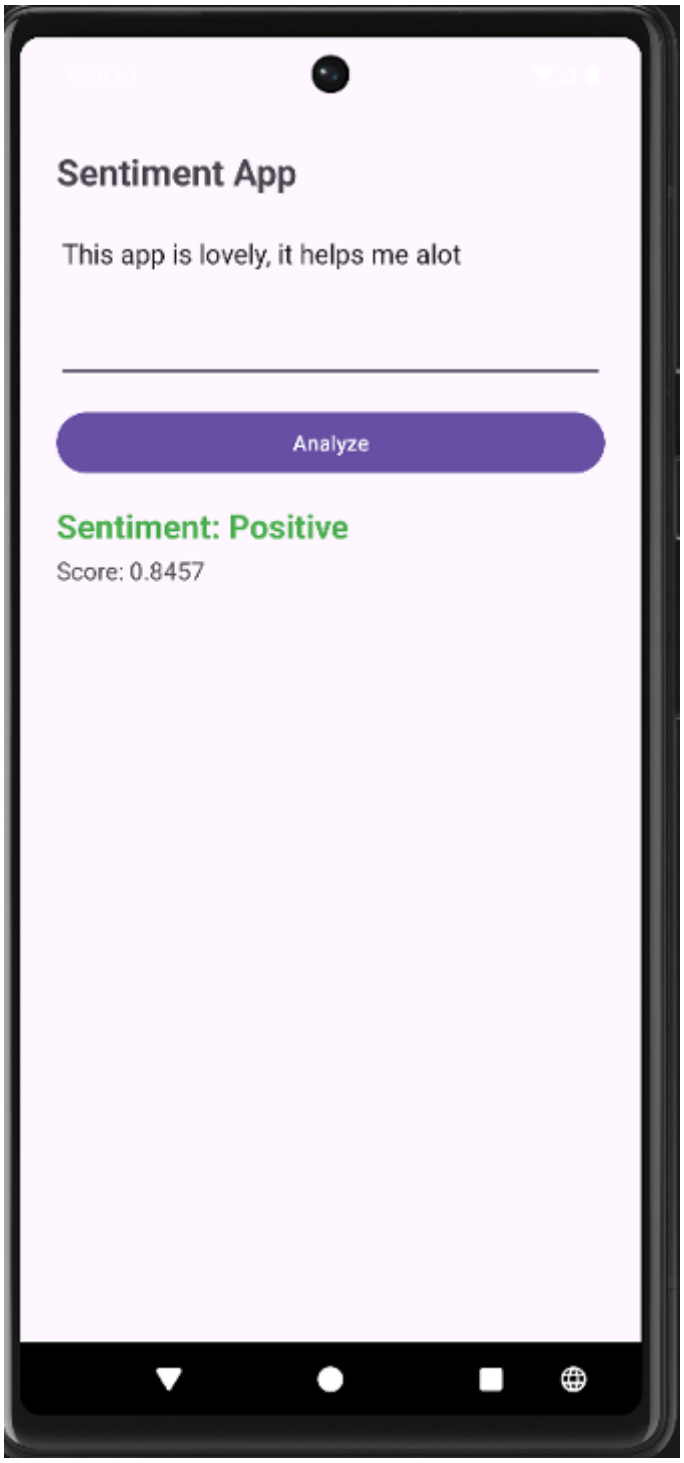
Không hiển thị score lên UI. Nếu result.getErrorMessage() khác null thì hiện tvError và set nội dung lỗi vào đó. Nếu không có lỗi thì ẩn tvError. Ngoài ra, đổi màu tvResult cho dễ nhìn: Positive màu xanh lá, Neutral màu xám hoặc cam, Negative màu đỏ, Unknown màu xám.

Trong file activity_main.xml, tạo layout đơn giản bằng LinearLayout vertical, padding khoảng 24dp. Layout gồm một TextView tiêu đề “Sentiment App”, một EditText cho người dùng nhập text, một Button Analyze, một TextView kết quả và một TextView lỗi. EditText nên có minLines khoảng 4, gravity top và inputType textMultiLine. TextView kết quả có text mặc định là “Sentiment: -”, textSize khoảng 22sp và in đậm. TextView lỗi để visibility gone mặc định và textColor màu đỏ. Không cần tạo TextView để hiển thị score.

App cần xử lý lỗi cẩn thận để không bị crash trong các trường hợp như người dùng chưa nhập text, thiếu file sentiment_model.tflite, model load lỗi, inference lỗi hoặc input/output shape chưa đúng. Nếu có lỗi thì tvResult vẫn hiển thị “Sentiment: Unknown”, còn tvError sẽ hiển thị message lỗi tương ứng.

Sau khi implement xong, hãy báo lại cho mình các file đã tạo hoặc sửa, các dependency đã thêm, và nhắc mình copy file sentiment_model.tflite vào đúng đường dẫn: app/src/main/assets/sentiment_model.tflite

Cuối cùng, hãy hướng dẫn ngắn cách chạy app: build project, run app, nhập một câu tiếng Anh, bấm Analyze và xem kết quả Positive, Neutral hoặc Negative.



- **News app:**

- Prompt: Mình là sinh viên đang học lập trình ứng dụng di động bằng Android Studio. Hiện tại mình đang làm bài homework News App, tên project/app là Homework1. Mình muốn bạn hỗ trợ tạo một app Android native bằng Java, dùng XML layout và ViewBinding. App này sẽ đọc tin tức từ NewsAPI và hiển thị danh sách bài báo dạng infinite scroll bằng Paging 3.
-
- Project cần dùng package là com.example.homework1, minSdk 24, compileSdk/targetSdk 36. Ngôn ngữ chính là Java, giao diện dùng XML layout truyền thống, AppCompat, Material Toolbar, ConstraintLayout và RecyclerView. Hãy bật ViewBinding trong Gradle để code dễ thao tác với view hơn. Mình muốn code rõ ràng, dễ hiểu, phù hợp với bài homework sinh viên nhưng vẫn build và chạy được.
-
- Mục tiêu chính của app là gọi NewsAPI để lấy danh sách tin tức, sau đó hiển thị các bài báo trong RecyclerView. API base URL là: https://newsapi.org/
- Endpoint dùng là: GET /v2/everything
- Query mặc định là “covid”. Các tham số API cần truyền gồm q, apiKey, page và pageSize. Phần API key không nên hard-code trực tiếp trong source code nếu có thể. Hãy dùng placeholder hoặc đọc từ cấu hình local, ví dụ local.properties hoặc BuildConfig, để tránh commit API key thật vào source.
- Về thư viện, hãy dùng Retrofit và Gson để gọi API. API service cần trả về ListenableFuture nên hãy dùng Retrofit Guava CallAdapter. Phần phân trang dùng AndroidX Paging 3 với ListenableFuturePagingSource. Ảnh bài viết load bằng Glide.
- Màn hình chính là MainActivity. Trong MainActivity hãy bật EdgeToEdge nếu phù hợp với project, sau đó inflate ActivityMainBinding. Màn hình có Material Toolbar với tiêu đề “News”, một RecyclerView để hiển thị danh sách tin tức và một ProgressBar nằm giữa màn hình. ProgressBar chỉ hiện khi app đang tải trang đầu tiên. MainActivity sẽ observe LiveData<PagingData<Article>> từ NewsViewModel rồi submit PagingData vào adapter.
- File activity_main.xml nên có root là ConstraintLayout với id main. Bên trên là AppBarLayout chứa MaterialToolbar. RecyclerView có id rvNews, nằm dưới toolbar và chiếm phần còn lại của màn hình. ProgressBar có id progressBar, đặt ở giữa màn hình và mặc định visibility là gone.
- Mỗi item bài viết nằm trong file item_article.xml. Mỗi item nên hiển thị trong một card cho dễ nhìn. Có ImageView id ivArticleImage, cao khoảng 200dp, scaleType centerCrop. Bên dưới là TextView tiêu đề id tvTitle, in đậm và maxLines 2. Tiếp theo là TextView mô tả id tvDescription, maxLines 3. Sau đó có TextView nguồn tin id tvSource và TextView ngày đăng id tvPublishedAt. Có thể dùng MaterialCardView, hoặc nếu dùng androidx.cardview.widget.CardView thì thêm dependency CardView tương ứng.
- Về model, hãy tạo các class Article, Source và NewsResponse. Article cần có các field title, description, urlToImage, publishedAt và source. Source có field name. NewsResponse có status, totalResults và List<Article> articles. Hãy dùng @SerializedName cho các field JSON để map dữ liệu từ API cho rõ ràng.

- Ở API layer, tạo interface NewsApiService. Trong đó có method getNews trả về ListenableFuture<NewsResponse>. Method này dùng annotation @GET("v2/everything") và các @Query gồm q, apiKey, page, pageSize.
- Phần Paging, hãy tạo NewsPagingSource extends ListenableFuturePagingSource<Integer, Article>. Trang đầu tiên là page 1. Trong loadFuture, lấy page hiện tại từ params.getKey(), nếu null thì dùng page 1. Sau đó gọi apiService.getNews(query, apiKey, page, params.getLoadSize()). Nếu response null hoặc articles null thì trả về LoadResult.Error. Nếu thành công thì trả về LoadResult.Page. prevKey là null nếu page == 1, ngược lại là page - 1. nextKey là null nếu danh sách articles rỗng, ngược lại là page + 1. Method getRefreshKey nên dựa trên anchorPosition, dùng closestPageToPosition để lấy prevKey/nextKey rồi tính refresh key phù hợp.
- Tạo NewsViewModel extends ViewModel. Trong ViewModel, khởi tạo Retrofit với baseUrl là https://newsapi.org/, thêm GsonConverterFactory và GuavaCallAdapterFactory. Sau đó tạo Pager<Integer, Article> với PagingConfig gồm pageSize = 20, prefetchDistance = 5 và enablePlaceholders = false. Expose LiveData<PagingData<Article>> bằng PagingLiveData.getLiveData(pager). Query mặc định là “covid”.
- Tạo NewsPagingAdapter extends PagingDataAdapter<Article, ArticleViewHolder>. Adapter dùng DiffUtil.ItemCallback<Article>. Trong onBindViewHolder, bind an toàn các dữ liệu gồm title, description, source name, publishedAt và urlToImage. Dùng Glide để load ảnh vào ivArticleImage. Nếu title, description, source hoặc image bị null thì app không được crash. Có thể hiển thị text mặc định như “No title”, “No description”, “Unknown source”, hoặc ẩn ảnh/hiển thị placeholder nếu không có image.
- Các dependencies cần có gồm appcompat, material, activity, constraintlayout, retrofit, converter-gson, adapter-guava, paging-runtime, paging-guava, glide, lifecycle-viewmodel, lifecycle-livedata và material card hoặc cardview nếu layout dùng card. Có thể thêm okhttp logging interceptor nếu cần debug API. Nếu dependency nào đã có sẵn thì không thêm trùng.
- Acceptance criteria của bài này là app phải build thành công bằng Gradle. Khi chạy app, màn hình có toolbar tiêu đề “News”. ProgressBar hiện khi tải dữ liệu đầu tiên. Danh sách tin tức được load từ NewsAPI và tự tải thêm khi scroll xuống cuối. Mỗi bài viết hiển thị ảnh, tiêu đề, mô tả, nguồn và ngày đăng. App không crash khi API trả thiếu title, description, source hoặc image. API key thật không được để trực tiếp trong commit/source code.
- Sau khi implement xong, hãy báo lại cho mình các file đã tạo hoặc sửa, dependency đã thêm, vị trí mình cần nhập NewsAPI key, và cách chạy app: thêm API key, sync Gradle, build project, run app và kiểm tra danh sách tin tức load ra màn hình.

10:20
News
BBC NEWS
Boy donates lockdown locks to children's charity
Noah braves the scissors and gets a short do after growing out his hair for six years.
BBC News
2026-04-14T13:24:04Z
BBC INDEPTH
BBC NEWS
Scammers are becoming ever more sophisticated - this is what the fightback look...
Scams have exploded over the last few years. Can countries and companies come together to turn the tables on the scammers?
BBC News
2026-04-18T23:02:48Z